



University of Engineering and Technology, Lahore

Department of Computer Science

Project Report

Team Members

Name	Roll No.	Email
Abdullah Ahmad	2025-CS-51	abdullahmubasher1710@gmail.com
Muhammad Aasher	2025-CS-16	aasher36890@gmail.com

Session: Fall'25 Morning

Section: A

Semester: 1st

Course: CSC101 - Discrete Mathematics

Teacher: Mr. Waqas Ali

January 04, 2026

Contents:

1. Project Title -----	03
2. Selected Track -----	03
3. Overview -----	03
4. Summary -----	03
5. Technology Stack & Libraries Utilized -----	04
6. List of Features -----	04
7. Mathematical Foundations -----	05
8. Examination of Key Function -----	06
9. Limitations -----	06
10. User Interface -----	07
11. Future Improvements -----	09
12. Conclusion -----	09
13. References & Learning Sources -----	09

1. Project Title

Relation Analyzer and Classifier

2. Selected Track

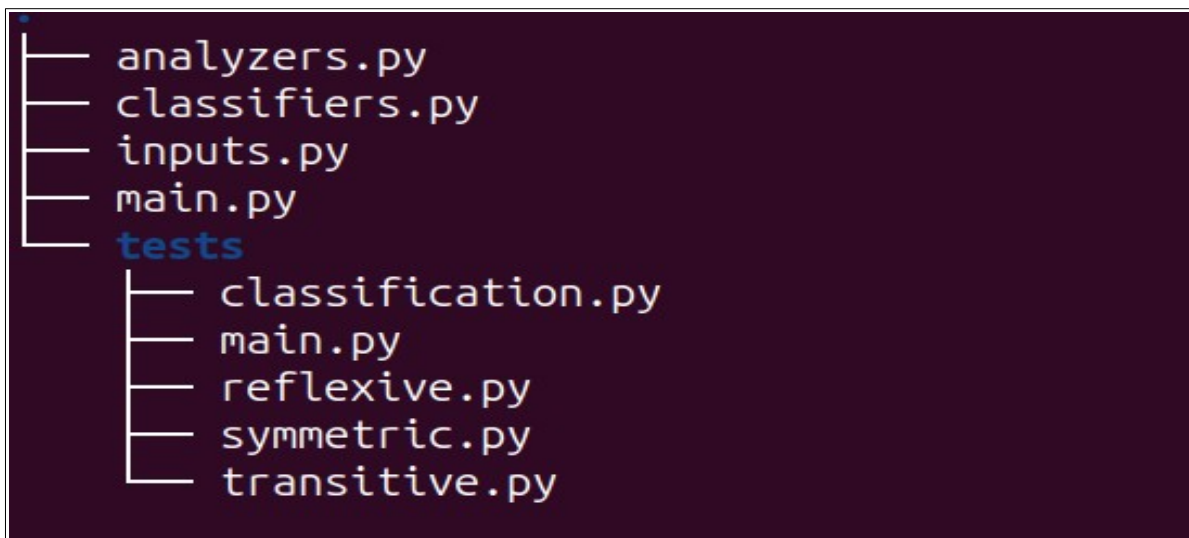
- ☐ Track 1: Foundations of Logic (Rosen Ch. 1–2)
- ☒ Track 2: Discrete Structures (Rosen Ch. 2, 9)
- ☐ Track 3: Algorithm Design & Analysis (Rosen Ch. 3)
- ☐ Track 4: Graph & Tree Algorithms (Rosen Ch. 10)
- ☐ Track 5: Number Theory Applications (Rosen Ch. 4–5)
- ☐ Track 6: Counting & Combinatorics (Rosen Ch. 6)

3. Overview

This project analyzes binary relations on a finite set to determine key properties such as reflexivity, symmetry, transitivity, equivalence, and partial order using a command line interface, directly mapping discrete mathematics theory into practical computation.

4. Summary

This project implements a Relation Analyzer based on fundamental concepts of Discrete Mathematics, focusing on the analysis and classification of binary relations defined on a finite set. The primary goal of the project is to bridge the gap between mathematical theory and practical implementation by translating formal definitions of relations into executable algorithms.



```
├── analyzers.py
├── classifiers.py
├── inputs.py
├── main.py
├── tests
│   ├── classification.py
│   ├── main.py
│   ├── reflexive.py
│   ├── symmetric.py
│   └── transitive.py
```

The system is modularly designed. The `inputs.py` module handles user interaction by collecting set elements and relation pairs through a command-line interface with proper validation. The `analyzers.py` module contains core functions that determine essential relation properties, including reflexive, irreflexive, symmetric, asymmetric, antisymmetric, and transitive characteristics. Based on these properties, the `classifiers.py` module identifies whether the given relation qualifies as an equivalence relation or a partial order. The `main.py` file integrates all modules and presents the analysis results in a clear, well-formatted tabular output.

To ensure correctness and reliability, the project includes a dedicated tests directory. This directory contains separate test files for reflexive, symmetric, transitive, and classification checks, along with a centralized test runner. All tests are implemented without using external testing frameworks, relying instead on logical verification and clear PASSED/FAILED outputs.

5. Technology Stack & Libraries Utilized

- Programming Languages: Python 3.14
- Libraries: Only Standard Libraries
- Development Environment: Standard Python IDLE

6. List of Features

- *Input Module:* Accepts a finite set and Relation on that set from the user via command line.

```
def get_set():
    elements = input("Enter Set Elements (Space Separated): ").strip()
    elements = elements.split()
    A = set(elements)
    return A

def get_relation(A):
    print_cartesian_product(A)
    while True:
        pairs = input("Enter Relation Pairs like (1,2) (2,3): ").strip()
        pairs = pairs.split()
        R = set()
        ready = True

        for pair in pairs:
            if not (pair.startswith("(") and pair.endswith(")")):
                print("Invalid Input\nPlease Follow the Instructed Format")
                ready = False
                break

            pair = pair[1:-1] # brackets remove
            pair = pair.split(",")

            if (len(pair) != 2):
                print("Invalid Input\nMore than 2 pair elements not allowed")
                ready = False
                break

            a = pair[0].strip()
```

- *Property Checker*: Determines reflexive, irreflexive, symmetric, asymmetric, antisymmetric, and transitive properties.

```
def is_reflexive(A, R):
    result = True
    for a in A:
        if (a, a) not in R:
            result = False
            break
    return result

def is_irreflexive(A, R):
    result = True
    for a in A:
        if (a, a) in R:
            result = False
            break
    return result

def is_symmetric(R):
    result = True
    for (a, b) in R:
        if (b, a) not in R:
            result = False
            break
    return result

def is_antisymmetric(R):
    result = True
    for (a, b) in R:
        if a != b and (b, a) in R:
            result = False
            break
    return result

def is_transitive(R):
    result = True
    for (a, b) in R:
        for (c, d) in R:
            if b == c and (a, d) not in R:
                result = False
                break
    return result
```

- *Relation Classification*: Identifies equivalence relations and partial orders.

```
from analyzers import (is_reflexive,
                       is_transitive,
                       is_symmetric,
                       is_antisymmetric)

def is_equivalence(A, R):
    if is_reflexive(A, R) and is_transitive(R) and is_symmetric(R):
        return True
    else:
        return False

def is_partial_order(A, R):
    if is_reflexive(A, R) and is_transitive(R) and is_antisymmetric(R):
        return True
    else:
        return False
```

- *Formatted Output*: Displays analysis results in a clear tabular format.

7. Mathematical Foundations

- Relevant Concepts

1. Sets and Cartesian Products
2. Binary Relations
3. Relation Properties (Reflexive, Symmetric, Transitive)

4. Equivalence Relations
5. Partially Ordered Sets

(Referenced from Rosen, Discrete Mathematics and Its Applications)

■ Formal Definitions

A relation R on a set A is a subset of $A \times A$.

1. Reflexive: $\forall a \in A, (a,a) \in R$
2. Irreflexive: $\forall a \in A, (a,a) \notin R$
3. Symmetric: $(a,b) \in R \Rightarrow (b,a) \in R$
4. Antisymmetric: $(a,b), (b,a) \in R \Rightarrow a=b$
5. Transitive: $(a,b), (b,c) \in R \Rightarrow (a,c) \in R$

A relation is a partial order if it is reflexive, antisymmetric, and transitive.

A relation is an equivalence if it is reflexive, symmetric, and transitive.

■ Proof of Correctness

Each checking function directly implements the formal mathematical definition by exhaustively verifying all required ordered pairs, ensuring correctness.

8. Examination of Key Functions

- *Function*: `check_transitive(R)`
- *Purpose*: Verifies whether the relation is transitive.
- *Algorithm*: For all (a,b) and (b,c) in R , check if (a,c) exists.
- *Time Complexity*: $O(n^2)$

```
def is_transitive(R):  
    result = True  
    for (a, b) in R:  
        for (c, d) in R:  
            if b == c and (a, d) not in R:  
                result = False  
                break  
    return result
```

9. Limitations

- Limited to finite sets of small to medium size
- Visualization becomes cluttered for large posets
- No support for weighted or infinite relations

- CLI-based interface only

10. User Interface

■ 10.1 Interface Overview

The project uses a command-line interface (CLI) designed for simplicity and clarity. Users are guided through set and relation input using prompts.

■ 10.2 Interface Workflow

1. Set Input: User enters set elements separated by spaces

```
Python 3.12.3 (main, Nov 6 2025, 13:44:16) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: /home/aasher/Uet 29'/1st sem/dm/project/main/main.py
Enter Set Elements (Space Separated): 1 2 3
```

2. Relation Input: User enters ordered pairs in a defined format

```
Cartesian Product (A x A):

(1, 1) (1, 3) (1, 2)
(3, 1) (3, 3) (3, 2)
(2, 1) (2, 3) (2, 2)

Enter Relation Pairs like (1,2) (2,3): (1,1) (2,2) (3,3) (1,2) (2,1)|
```

3. Analysis Output: Program displays relation properties and classification in tabular form

```
Relation Analysis:
-----
Property                                Result
-----
Reflexive                               Yes
Irreflexive                             Not
Symmetric                               Yes
Asymmetric                              Not
Antisymmetric                           Not
Transitive                              Yes
-----

Relation Classification:
-----
Property                                Result
-----
Equivalence                             Yes
Partial Order                           Not
-----
>>>
```

■ 10.3 Sample Test Case

1. Input: $A = \{1,2,3\}$, $R = \{(1,1),(2,2),(3,3),(1,2), (2,1)\}$

2. Expected Output:

Reflexive	:	Yes
Irreflexive	:	Not
Symmetric	:	Yes
Asymmetric	:	Not
Antisymmetric	:	Not
Transitive	:	Yes
Equivalence	:	Yes
Partial Oeder	:	Not

3. Actual Output: Matches expected result

4. Status: Passed

```

Enter Set Elements (Space Separated): 1 2 3
Cartesian Product (A x A):

(1, 1) (1, 3) (1, 2)
(3, 1) (3, 3) (3, 2)
(2, 1) (2, 3) (2, 2)

Enter Relation Pairs like (1,2) (2,3): (1,1) (2,2) (3,3) (1,2) (2,1)

Relation Analysis:
-----
Property          Result
-----
Reflexive          Yes
Irreflexive        Not
Symmetric          Yes
Asymmetric         Not
Antisymmetric      Not
Transitive         Yes
-----

Relation Classification:
-----
Property          Result
-----
Equivalence        Yes
Partial Order      Not
-----

```


11. Future Improvements

- GUI-based interface
- Generating Hasse diagram
- Export diagrams as image files

12. Conclusion

This project effectively applied discrete mathematics principles to analyze and classify binary relations through a structured Python implementation. It strengthened understanding of relation properties and demonstrated how mathematical theory directly guides correct algorithm design. Overall, the project reflects improved analytical thinking and a clear connection between theoretical concepts and practical computation.

13. References & Learning Sources

- Rosen, K. H. *Discrete Mathematics and Its Applications*, Chapters 2 and 9.
- Python Official Documentation: <https://docs.python.org>
- ChatGPT was used for structuring explanations and improving clarity
- Github Repo: <https://github.com/Aasher-3689/relation-analyzer>
- Demo Video: <https://youtu.be/4BrXpQseAfU>